

Docker: A 7-Day Hands-On Guide

A practical, step-by-step companion to the Docker & Containers slides. Each day pairs a few core ideas with commands you run yourself. The goal is simple: by the end you can package an app, ship it to a registry, and run it anywhere — reliably.

Read the idea, run the command, watch what happens. Don't just copy-paste; predict the result first, then check whether you were right.

Before you start: session setup

Everyone in this class shares **one Docker daemon on one machine**. Docker keeps containers separate from the host, but it does **not** keep your work separate from your classmates'. If two of you both name a container `web` or both publish to port `8080`, you'll collide. So at the start of **every session**, claim your own identity:

```
bash

# Run this once each time you log in.
export DKR=$USER                    # your personal prefix for everything
export PB=$(( 20000 + (UID % 800) * 5 )) # your five private ports: PB to PB+4
export COMPOSE_PROJECT_NAME=$DKR      # keeps your Compose resources separate
echo "You are: $DKR | Your ports: $PB to $((PB+4))"
```

Then follow three rules the whole week:

1. **Name everything after yourself.** Containers `--name $DKR-web`, images `$DKR/app:1`, networks `$DKR-net`, volumes `$DKR-data`. Also tag containers with `--label owner=$DKR` so you can find your own later.
2. **Use only your own ports** (`$PB` to `$PB+4`).
3. **Only delete your own things.** Never run `docker system prune` or remove a shared base image — that wipes other people's work too. To clean up just yours:

```
bash

docker rm -f $(docker ps -aq --filter "label=owner=$DKR") 2>/dev/null
docker volume rm $DKR-data 2>/dev/null; docker network rm $DKR-net 2>/dev/null
```

Day 1 — Foundations

Slides: Why containers · What a container is · Containers vs VMs · The ecosystem · Architecture

Today you don't build anything. You prove to yourself what a container actually is.

Key ideas

- A **container** is like a shipping container: your app plus everything it needs, sealed into a standard box that runs the same on any machine with a runtime.
- A container **shares the host's kernel**. A virtual machine brings its own entire operating system instead — that's why a VM is measured in gigabytes and starts in seconds, while a container is megabytes and starts in milliseconds.
- **Namespaces** are the walls that give a container its own isolated view. **cgroups** are the limits on how much CPU and memory it's allowed to use.

Commands you'll use

`docker run`, `docker version`, `docker info`, `uname -r`, `ps`, and the flags `--rm`, `--memory`, `--cpus`.

Steps

1.1 — Run your first container

```
bash
docker run --rm hello-world
```

Read every line it prints. In your own words, trace what happened: the client sent a command, the daemon didn't have the image, it pulled it, created a container, ran it, and it exited. That sentence is the whole architecture diagram.

1.2 — Prove the kernel is shared

```
bash
uname -r                # the host's kernel
docker run --rm alpine uname -r    # the container's kernel
```

They are identical. There is no separate operating system inside the container. *Hint: this one fact is the entire difference between a container and a VM — make sure you can explain it.*

1.3 — See the isolation

```
bash
docker run --rm alpine ps aux    # the container sees only its own process
ps aux | head                    # the host sees hundreds
```

Same kernel, completely different view. That different view is a namespace.

1.4 — See the limits

```
bash
docker run --rm --memory=64m alpine sh -c 'echo "capped at 64MB RAM"'
docker run --rm --cpus=0.5 alpine sh -c 'echo "capped at half a CPU"'
```

Namespaces decide what a container can see; cgroups decide how much it can *take*.

1.5 — Look at the runtime stack

```
bash
```

```
docker info | grep -i runtime
```

You'll see `runc`. The chain is: your `docker` command → the `dockerd` daemon → `containerd` → `runc` → the kernel. *Hint: Kubernetes stopped using Docker as its runtime and talks to containerd directly — and your images still run fine, because they follow the OCI standard.*

If you finish early

Run `docker run --rm -it alpine sh`, then inside type `cat /etc/os-release`. You appear to be "inside Alpine" even though the host is Ubuntu. Work out how that's possible with only one shared kernel. (Hint: it's a different set of files, not a different engine.)

Check yourself

1. Why does a container start in milliseconds but a VM in seconds?
2. What does a VM include that a container deliberately leaves out?
3. Which kernel feature provides isolation, and which one limits resources?

Common mistake

"A container is just a lightweight VM." It isn't. A VM virtualises hardware and runs its own OS; a container isolates a process and shares the host kernel. If there's no shared kernel, it isn't a container.

Day 2 — Running Containers

Slides: Installing Docker · Image to container · The lifecycle · `docker run` · Publishing ports · Inspect and clean up

Now you operate containers by hand: pull an image, run it, expose it, look inside, remove it.

Key ideas

- An **image** is a read-only template; a **container** is a running instance of that image. Think of the image as a recipe and the container as the dish, or the image as a class and the container as an object. One image can produce many containers.
- A running container has a thin **writable layer** on top. Anything written there is lost when the container is removed.
- **Publishing a port** with `-p` is what connects the outside world to a port inside the container. Without it, nothing can reach the app.

Commands you'll use

```
docker pull, docker images, docker run (-d --name -p -e -v -it --rm),  
docker ps -a, docker logs -f, docker exec -it, docker stop/start/pause/  
unpause, docker rm/rmi.
```

Steps

2.1 — Pull a specific image

```
bash
```

```
docker pull nginx:1.27
docker images | grep nginx
```

Hint: notice we pinned `:1.27` instead of `:latest`. `:latest` quietly changes over time, which makes builds impossible to reproduce.

2.2 — Run it in the background and reach it

```
bash
```

```
docker run -d --name $DKR-web -p $PB:80 --label owner=$DKR nginx:1.27
curl -s localhost:$PB | head -n 4
```

You published your own port (`$PB`) to port (`80`) inside the container.

2.3 — Walk the lifecycle

```
bash
```

```
docker pause    $DKR-web && docker ps      # look for the Paused state
docker unpause  $DKR-web
docker stop     $DKR-web && docker ps -a    # gone from `ps`, still listed in `
docker start    $DKR-web
```

Match each command to the lifecycle diagram: Created, Running, Paused, Stopped, Removed.

2.4 — Look inside a running container

```
bash
```

```
docker exec -it $DKR-web bash
# inside: ls /usr/share/nginx/html ; exit
docker logs --tail 5 $DKR-web
```

`exec` opens a shell inside a container that's already running. `logs` shows what it has printed.

2.5 — One image, many containers

```
bash
```

```
for n in 1 2 3; do
  docker run -d --name $DKR-web-$n -p $((PB+n)):80 --label owner=$DKR nginx:1
done
docker ps --filter "label=owner=$DKR"
```

Three identical containers from one image. This is scaling, done by hand for now.

2.6 — Clean up your own containers

```
bash
```

```
docker rm -f $(docker ps -aq --filter "label=owner=$DKR")
docker ps -a --filter "label=owner=$DKR"      # should be empty
```

Hint: you removed only the containers labelled with your name. Get used to this — it's how you stay out of your classmates' way.

If you finish early

Run a throwaway container: `docker run -it --rm ubuntu bash`. Make a mess inside, then `exit`. Confirm with `docker ps -a` that `--rm` left nothing behind. When would you want that instead of `-d`?

Check yourself

1. What three things does `docker run` do in order?
2. A stopped container still uses something — what, and how do you reclaim it?
3. In `-p 8080:80`, which number is the host port and which is the container port?

Common mistake

`EXPOSE` in a Dockerfile does not publish a port — only `-p` does. Also remember `docker rm` removes a *container* but leaves the *image* on disk; `docker rmi` removes the image.

Day 3 — Building Images with a Dockerfile

Slides: Hand-built containers don't scale · Anatomy of a Dockerfile · Instruction reference · CMD vs ENTRYPOINT

So far you've run images other people built. Now you write the instructions to build your own, so that anyone can rebuild exactly the same image.

Key ideas

- A **Dockerfile** is a plain-text recipe for an image. It lives in git, so it's reviewable and repeatable. The rule to remember: *if it isn't in the Dockerfile, it doesn't exist*.
- **CMD** sets the default command, and it's easy to override at run time.
ENTRYPOINT sets the program that always runs; anything you pass on the command line becomes arguments to it.

Commands you'll use

`docker build -t`, and the instructions `FROM WORKDIR COPY RUN ENV EXPOSE USER CMD ENTRYPOINT`, plus a `.dockerignore` file.

The app you'll build

Make a folder `~/ $DKR -app` and put this file in it. It uses only Python's standard library, so there's nothing to install:

```
python

# server.py
import http.server, os
port = int(os.environ.get("PORT", "8000"))
msg = os.environ.get("GREETING", "Hello from my container")
class H(http.server.BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200); self.end_headers()
        self.wfile.write(f"{msg} (port {port})\n".encode())
print(f"serving on {port}")
http.server.HTTPServer(("", port), H).serve_forever()
```

Steps

3.1 — Write and build your first image

```
dockerfile

# Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY server.py .
ENV PORT=8000
EXPOSE 8000
CMD ["python", "server.py"]
```

```
bash

cd ~/$DKR-app
docker build -t $DKR/app:1 .
docker run -d --name $DKR-app -p $PB:8000 --label owner=$DKR $DKR/app:1
curl -s localhost:$PB
```

Hint: watch the build output. Each instruction becomes a layer — that detail matters on Day 4.

3.2 — Add a `.dockerignore`

```
bash

printf '.git\n__pycache__\n*.log\nREADME.md\n' > .dockerignore
```

Everything in the folder is sent to the daemon as the "build context." Keep junk and secrets out of it.

3.3 — Override CMD

```
bash

docker run --rm -e GREETING="custom message" $DKR/app:1
docker run --rm $DKR/app:1 python -c "print('I replaced the CMD')"
```

The second command replaced CMD entirely. That's how easily CMD is overridden.

3.4 — Compare with ENTRYPOINT Build a second version that pins the program:

```
dockerfile

# Dockerfile.entry
FROM python:3.12-slim
WORKDIR /app
COPY server.py .
ENTRYPOINT ["python", "server.py"]
```

```
bash

docker build -f Dockerfile.entry -t $DKR/app:entry .
docker run --rm $DKR/app:entry --help      # the arguments go TO python, they
```

With ENTRYPOINT, command-line arguments are appended to the program instead of replacing it. *Hint: use CMD when you want users to swap the command easily; use ENTRYPOINT when the program should always be the same and only its arguments change.*

3.5 — Stop running as root Add these two lines before `CMD` in your first Dockerfile, then rebuild:

```
dockerfile

RUN useradd -m appuser
USER appuser
```

```
bash

docker build -t $DKR/app:1 .
docker run --rm $DKR/app:1 whoami      # should print appuser, not root
```

If you finish early

Break the build on purpose: point a `COPY` at a file that doesn't exist. Read the error message carefully, then fix it. Reading a failed build is half the skill.

Check yourself

1. Why is a Dockerfile better than configuring a container by hand and saving it?
2. You want users to be able to swap the command easily — CMD or ENTRYPOINT?
3. What is the "build context," and why does `.dockerignore` matter?

Common mistake

Use the bracket form: `CMD ["python", "server.py"]`, not `CMD python server.py`. The bracket ("exec") form runs your program directly; the plain form wraps it in a shell and can break how the container handles stop signals.

Day 4 — Smaller, Faster, Safer Images

Slides: Layers and the build cache · Best practices · Multi-stage builds · Security hardening

Anyone can build an image. Today you make one that rebuilds quickly, stays small, and doesn't run as root.

Key ideas

- Each instruction adds a **layer**, stacked like pallets. Change a lower pallet and every pallet above it has to be re-stacked — that's a cache miss cascading down.
- So the ordering rule is: **install dependencies first, copy your changing source code last**. Then editing your code never forces the slow dependency install to run again.
- A **multi-stage build** compiles in a big image but ships only the finished artifact in a tiny one.

Commands you'll use

`docker build` (watch for the word `CACHED`), multi-stage builds with `FROM ... AS build` and `COPY --from=build`, `docker images` to compare sizes, and `time`.

Steps

4.1 — Watch the cache work

```
bash

docker build -t $DKR/app:1 .           # first build: every step runs
docker build -t $DKR/app:1 .           # nothing changed: every step says CACHED
touch server.py                        # pretend you edited the source
docker build -t $DKR/app:1 .           # only the COPY step and below rebuild
```

Hint: seeing `CACHED` appear and disappear is the whole point. This is why instruction order matters.

4.2 — Break the cache the wrong way Move `COPY server.py .` to be the first line right after `FROM`, then rebuild twice with an edit in between. Now unrelated steps rebuild too. The lesson: put rarely-changing steps first.

4.3 — Shrink an image with a multi-stage build A compiled language makes the size difference dramatic. Put these two files in a new folder:

```
go

// main.go
package main
import ("fmt"; "net/http")
func main() {
    http.HandleFunc("/", func(w http.ResponseWriter, _ *http.Request) { fmt.F
    http.ListenAndServe(":8000", nil)
}
```


dockerfile

```
# Dockerfile
FROM golang:1.22 AS build
WORKDIR /src
COPY main.go .
RUN go build -o app main.go

FROM gcr.io/distroless/static
COPY --from=build /src/app /app
USER nonroot
ENTRYPOINT ["/app"]
```

bash

```
docker build -t $DKR/tiny:1 .
docker images | grep -E "golang|$DKR/tiny" # compare the builder vs the f
```

The final image contains only the compiled program — no compiler, no shell, no extra OS files. That gap is the entire reason multi-stage builds exist.

4.4 — Compare base image sizes

bash

```
docker pull python:3.12 ; docker pull python:3.12-slim ; docker pull python:3
docker images | grep python
```

Same language, very different sizes. Fewer packages means a smaller image and a smaller attack surface.

4.5 — Scan for vulnerabilities *(only if the tool is installed)*

bash

```
docker scout cves $DKR/app:1 # or: trivy image $DKR/app:1
```

Find a reported vulnerability, rebuild on an updated base image, and scan again. This is exactly the check a real pipeline runs before shipping (you'll see it on Day 7).

If you finish early

Take your Day 3 Python image and shrink it as far as you can: a `-slim` base, a `.dockerignore`, a non-root `USER`, and combined `RUN` steps. Compare your final `docker images` size with a classmate's.

Check yourself

1. You changed one line of source. Why did the dependency install *not* run again (assuming your Dockerfile is ordered well)?
2. What does a multi-stage build leave out of the final image, and why is that good?
3. Name three things from the security slide you'd check before shipping an image.

Common mistake

Putting `COPY . .` near the top "to be safe." It busts the cache on every code change and makes rebuilds slow. The order is: dependency files, then install, then the rest of the source last.

Day 5 — Networking and Storage

Slides: How containers get on the network · Network drivers · Connecting by name · Containers forget everything · Volumes, bind mounts, tmpfs

Two problems today. How do two containers talk to each other? And how do you keep data when a container is removed?

Key ideas

- On a **user-defined network**, containers reach each other **by name** through Docker's built-in DNS, instead of by an IP address that changes on every restart.
- A container's writable layer is wiped when the container is removed. A **volume** is storage that lives outside the container, survives removal, and can be shared between containers. The rule: *code belongs in the image; data belongs in a volume.*

Commands you'll use

`docker network create/ls`, `--network`, `docker volume create/ls`, `-v name:/path`, a bind mount with `-v $(pwd):/path`, and `--tmpfs`.

Steps

5.1 — Reach a container by name

```
bash
docker network create $DKR-net
docker run -d --name $DKR-db --network $DKR-net --label owner=$DKR alpine sl
docker run --rm --network $DKR-net alpine getent hosts $DKR-db
```

That last command resolved `$DKR-db` to an IP by name — no IP address was ever hard-coded. In a real app, your API would simply connect to `postgres://$DKR-db:5432`.

5.2 — Show that the default network can't do this

```
bash
docker run --rm alpine getent hosts $DKR-db # not on your network, so no r
```

The default bridge has no name-based DNS. This is why you always create your own network.

5.3 — Keep data with a volume

```
bash
docker volume create $DKR-data
docker run --rm -v $DKR-data:/out alpine sh -c 'echo "version 1" > /out/log.txt'
docker run --rm -v $DKR-data:/out alpine cat /out/log.txt # still there
```

The container that wrote the file is gone, but the file remains. That's a volume.

5.4 — Lose data without one

```
bash
```

```
docker run --name $DKR-tmp alpine sh -c 'echo secret > /tmp/note'
docker rm $DKR-tmp
```

The file was on the writable layer, so removing the container destroyed it. *Hint: compare this with 5.3 — that contrast is the most important storage lesson of the week.*

5.5 — Edit files live with a bind mount

```
bash
```

```
mkdir -p ~/$DKR-site && echo "<h1>edit me live</h1>" > ~/$DKR-site/index.html
docker run -d --name $DKR-live -p $((PB+1)):80 --label owner=$DKR \
  -v ~/$DKR-site:/usr/share/nginx/html:ro nginx:1.27
curl -s localhost:$((PB+1))
echo "<h1>changed on the host</h1>" > ~/$DKR-site/index.html
curl -s localhost:$((PB+1))          # changes immediately, no rebuild
```

A bind mount maps a folder on the host straight into the container — useful while developing.

5.6 — Memory-only storage with tmpfs

```
bash
```

```
docker run --rm --tmpfs /scratch:rw,size=16m alpine sh -c 'echo ram-only > /s
```

This never touches disk. It's for secrets and scratch data, and it disappears when the container stops.

If you finish early

From the network drivers table on the slides, write a one-line use case for each of `bridge`, `host`, `none`, `overlay`, and `macvlan`. Then run `docker run --rm --network none alpine ip addr` and confirm the container has no network.

Check yourself

1. Why connect to another container by name instead of by IP?
2. You remove a database container. Was its data on the writable layer or a volume? How do you make sure it survives?
3. Volume, bind mount, tmpfs — give the one-line use case for each.

Common mistake

Cleanup needs extra steps today. Removing your containers doesn't remove your network or volume — those need `docker network rm $DKR-net` and `docker volume rm $DKR-data`. Left-behind volumes are the quiet way a shared machine runs out of disk.

Day 6 — Registries and Compose

Slides: Docker registry and Hub · Self-hosted registry and Harbor · Real apps are many containers · A compose file · Compose beyond the basics

Two skills: send an image to a registry so other machines can pull it, and run a whole multi-container app from a single file.

Key ideas

- A **registry** stores and serves images, addressed by `name:tag`. A **digest** (`@sha256:...`) locks the exact bytes, so it can't change underneath you.
- **Compose** describes a whole application — every service, network, and volume — in one YAML file. `docker compose up` starts the entire stack together, instead of many separate `docker run` commands in the right order.

Commands you'll use

`docker tag/push/pull`, `docker login`, and `docker compose up -d / ps / down / logs / exec`.

Your instructor is running a class registry at `localhost:5000`. You'll push to it instead of Docker Hub, so you don't need an account.

Steps

6.1 — Push your image to the registry

```
bash
docker tag $DKR/app:1 localhost:5000/$DKR/app:1
docker push localhost:5000/$DKR/app:1
```

Hint: note the `sha256` digest it prints back — that's the exact-bytes fingerprint of your image.

6.2 — Pull it back fresh

```
bash
docker rmi localhost:5000/$DKR/app:1 # remove your LOCAL copy
docker run --rm localhost:5000/$DKR/app:1 echo "pulled from the registry"
```

You deleted the local image and Docker fetched it from the registry. Your image now lives somewhere portable, not just on this machine.

6.3 — Describe a whole stack in one file In `~/ $DKR-app`, create `compose.yaml`:

```
yaml
services:
  web:
    build: .
    ports: ["${PB}:8000"]
    environment:
      GREETING: "served by the stack"
    depends_on: [cache]
  cache:
    image: redis:7-alpine
```

```
bash
```

```
PB=$PB docker compose up -d
docker compose ps
curl -s localhost:$PB
```

Because you set `COMPOSE_PROJECT_NAME=$DKR` during setup, all of your Compose resources are prefixed with your name and won't collide with anyone else's. Compose also creates a network for the project automatically, which is why `web` can reach `cache` by name.

6.4 — Operate the stack

```
bash
```

```
docker compose up -d --scale web=3          # three copies of web, one command
docker compose logs --tail 20 web
docker compose exec cache redis-cli ping    # PONG
docker compose down                        # stops and removes the whole stack
```

Hint: remember the `for` loop you wrote on Day 2 to run three containers? Compose just did the same thing declaratively, and tears it all down with one command.

If you finish early

Add a `healthcheck` to the `cache` service, and make `web` depend on it with `condition: service_healthy`. Show that `web` now waits until Redis is actually answering, not just started.

Check yourself

1. What does a digest guarantee that a tag does not?
2. Inside a Compose project, how does `web` find `cache`? What did you not have to set up?
3. What's the difference between `docker compose down` and `docker compose stop`?

Common mistake

`depends_on` controls start order only, not readiness. A database can be "started" but not yet accepting connections. The fix is a `healthcheck` plus `condition: service_healthy`. Also: run `docker compose down` at the end of every session so you don't leave stacks running.

Day 7 — Pipelines and Orchestration

Slides: Deploy via a CI/CD pipeline · One host is not enough · Docker Swarm · Swarm vs Kubernetes · The Docker workflow

Everything connects today: build, ship, and run, automatically and at scale.

Key ideas

- A **CI/CD pipeline** is just the commands you already know — build, test, scan, push, deploy — run automatically on every commit, with a gate in the middle. If the security scan finds a critical vulnerability, the build fails and nothing ships.
- An **orchestrator** keeps a target number of containers running across machines. You declare "I want three of these"; it schedules them, balances traffic, and restarts any that fail. You state the desired result, and it maintains it.

Commands you'll use

`docker scout` or `trivy` with a fail-on-critical flag, a small shell script, and (in the demonstration) `docker swarm init` and `docker service create/scale/update/ps`.

Steps

7.1 — Build the pipeline as a script

A pipeline is these steps in order. Save this as `deploy.sh` in `~/Docker-app` and run it:

```
bash

#!/usr/bin/env bash
set -euo pipefail
SHA=$(date +%s) # stands in for a git commit ID
REG=localhost:5000/$DKR/app

echo "build"; docker build -t $REG:$SHA .
echo "test"; docker run --rm $REG:$SHA python -c "print('tests pass')"
echo "scan"; docker scout cves --exit-code --only-severity critical $REG:$SHA
              || { echo "critical vulnerabilities found — stopping, nothing shipped"; exit 1; }
echo "push"; docker push $REG:$SHA
echo "deploy"; SHA=$SHA docker compose up -d
echo "shipped $REG:$SHA"
```

```
bash

chmod +x deploy.sh && ./deploy.sh
```

Hint: CI/CD isn't a special tool — it's discipline written as a script. The scan step is a gate: try pointing your Dockerfile at an old, vulnerable base image and watch the deploy never run. (If the scan tool isn't installed, replace that line with a placeholder and note where the real gate goes.)

7.2 — A self-healing cluster (instructor demonstration)

A single machine runs only one cluster, so you'll watch this rather than run it yourself. Follow along on the projector:

```
bash

docker swarm init
docker service create --name fleet --replicas 3 -p 8080:80 nginx:1.27
docker service ps fleet # three tasks scheduled

docker rm -f $(docker ps -q --filter name=fleet | head -1) # kill one
docker service ps fleet # it's rescheduled automatically

docker service scale fleet=5 # scale up
docker service update --image nginx:1.27 fleet # rolling update
```

The key idea to take away: you declared three replicas; one was killed; it stayed three. You state the desired result, and the orchestrator keeps it true.

7.3 — Swarm versus Kubernetes Using the slide's comparison: Swarm is built into Docker and simple to learn; Kubernetes is the industry standard at scale, far more powerful, and harder to learn. *Hint: the images you built this week run unchanged on either one — orchestration changes how containers are managed, not what's inside them. Kubernetes is the next module.*

7.4 — Say the whole workflow from memory

Write a recipe (Dockerfile), build an image, ship it through a registry and a pipeline, run it anywhere — from a single container to a cluster.

If you can say that and explain each part, you've understood the module.

If you finish early

Turn `deploy.sh` into a real `.github/workflows/deploy.yml`. Match each shell line to a workflow step. You don't have to run it — translating it proves you understand what CI is doing.

Check yourself

1. Why is the scan a *gate* rather than just a report?
2. When a container dies, what does the orchestrator actually do, and why?
3. Do your images need to change to run on Kubernetes? Why or why not?

Common mistake

Thinking CI/CD is a separate, magical tool. It's the same `docker build`, `push`, and `run` commands you already know, run automatically on every commit, with a scan as the gate.

Where you are now

You can package an application into an image, make that image small and non-root, store and share it through a registry, run a multi-container stack from one file, and put the whole thing behind an automated pipeline that ships to a cluster.

The build-ship-run cycle is the same at every scale. Next up is Kubernetes, which runs these exact images across many machines.